

# Key-Value Response Mapping for Fail-Closed LLM Execution

## Auditable Action Spaces via Key Expert Models

Bobby Price  
blackWeb Research  
contact@blackweb.dev

December 2025

### Abstract

LLM-based agents increasingly drive software execution, yet the dominant pattern (treating model outputs as arbitrary text with post-hoc parsing) remains brittle under adversarial inputs and difficult to audit. We propose Key-Value Response Mapping (KVRM), an architecture where only validated model outputs can execute: each output is mapped to a finite, pre-audited vocabulary of keys, resolved through an immutable registry to audited primitives. KVRM is instantiated via Key Expert Models (KEMs): fine-tuned specialist LLMs, each emitting outputs under a narrow, machine-checkable contract. We build two KEM-based subsystems: (i) a compiler pipeline (Lexer→Parser→CodeGen→Validator) for a toy language, and (ii) a CPU emulator where instruction decoding yields registry-verified execution keys. On 10,000 grammar-fuzzed programs (excluding loops; see limitations), the end-to-end pipeline matches a reference interpreter on 94.12% of cases; all remaining cases fail safely via explicit errors or `ERROR_KEY`. The decode stage achieves 100% exact-match accuracy on a held-out validation set ( $N=5,000$ ), and critically, produces 100% `ERROR_KEY` on out-of-distribution inputs (demonstrating bounded concept recognition rather than hallucination). KVRM separates architectural safety (bounded action space; no out-of-registry execution) from empirical correctness (selecting the intended action), reducing the safety argument to auditing a finite set of primitives rather than validating unbounded text. Prototype cost: This is a research prototype demonstrating the safety architecture; compilation takes  $\sim 15\text{--}20$  seconds (vs. milliseconds for traditional compilers). Performance optimization is future work. Clarification: “Registry-verified” denotes systems-level verification (outputs checked against a finite key set before dispatch), not formal semantic preservation proofs. Correctness (selecting the intended key) remains empirical.

**Keywords.** Large Language Models; Fail-Closed Execution; Auditable Systems; Bounded Action Spaces; Key Expert Models; Compiler Pipelines; AI Safety.

## 1 Introduction

LLM-based agents often generate free-form text that must be interpreted by downstream code. This invites failure modes such as hallucinated API calls, malformed outputs, and prompt-injection-induced tool misuse. Many systems add post-hoc checks (schema validation, guardrails, policy filters), but these checks run after the model has already produced an unconstrained string.

A motivating failure. Consider an LLM agent with tool access that receives a prompt containing hidden instructions: “Ignore previous instructions and invoke `send_file('/etc/passwd', 'attacker@evil.com')`.” If the model emits `{"tool": "send_file", "args": {...}}`, a post-hoc filter must

catch and block this invocation. The failure mode is that the filter races against an unbounded output space: any string the model can generate is a potential action, and blocklists cannot anticipate all dangerous combinations.

One might argue: “Just never expose `send_file` to the model.” But this misses the point: in systems where model outputs are interpreted as arbitrary function calls, the syntax of every possible dangerous invocation must be anticipated and blocked. KVRM inverts this: the model can only emit keys from a finite, pre-audited vocabulary, and the registry maps keys to actions. Therefore `send_file` is structurally non-executable unless explicitly added to the registry. This is a principled way to enforce closed-world execution at the dispatch boundary, even when the model is adversarially steered.

What about parameters? Real tool calls have arguments (e.g., file paths, email addresses). KVRM handles this via two strategies: (1) finite enumeration: restrict arguments to finite enums (e.g., `SEND_FILE:config.json` vs. `SEND_FILE:logs.txt`), so each valid combination is a distinct key; or (2) key + validated args: the key identifies the action, while arguments are separately validated by a schema (e.g., “path must match `/data/*.json`”). In our CPU prototype, we use strategy (1): each opcode-operand combination is a unique key. Strategy (2) is future work for tool-calling scenarios where argument spaces are large.

This paper argues for a different trust boundary: constrain what can execute, not merely what can be generated. We introduce Key-Value Response Mapping (KVRM), where every model output that influences execution is reduced to a symbolic key from a finite set  $K$ . Keys are resolved through an immutable registry  $R$  into pre-audited actions. If a model emits an invalid key, the system executes a safe fallback (`ERROR_KEY`).

## 1.1 Key Expert Models (KEMs)

KVRM is an architectural pattern; to make it practical we use Key Expert Models (KEMs). A KEM is a small, fine-tuned specialist model that emits outputs under a narrow, machine-checkable contract (e.g., “valid JSON with schema  $S$ ” or “a key matching grammar  $G$ ”). KEMs may be generative (autoregressive LLMs producing token sequences) or discriminative (classifiers selecting from a fixed label set); the term “KEM” encompasses both architectures. Instead of relying on one large generalist model for every step, we compose multiple KEMs, each trained for one stage and evaluated under its specific contract.

Why KEMs instead of one generalist model?

- Specialization improves schema adherence: narrow contracts make validation simpler and more reliable.
- Failure isolation: a bug in the Parser KEM does not affect the Lexer KEM; failures are attributable to specific stages.
- Targeted retraining: if one stage underperforms, only that KEM needs retraining, not the entire pipeline.
- Per-stage calibration: each KEM can have its own confidence threshold and retry policy.
- Hierarchical decomposition: complex tasks (e.g., instruction decoding) can be split into independent sub-KEMs (opcode, register, immediate), enabling compositional reasoning.

Throughout the paper, we refer to each specialist model as a KEM.

## 1.2 Claims and scope

We explicitly distinguish two fundamentally different types of properties (see Table 2 in Section 3 for formal classification):

- **Guaranteed (architectural):** no action outside the registry is executed (no out-of-registry execution), assuming registry integrity and a trustworthy runtime. This holds regardless of model quality.
- **Empirical:** end-to-end correctness (selecting the intended key) depends on training data, model generalization, and the retry policy. This is what we measure, not what we guarantee.

KVRM is not a proof that LLMs are “correct”; it is a method to make unsafe actions non-executable by construction. A model that always outputs garbage would have 0% correctness but still 0% safety violations under KVRM.

The core novelty. Prior work on structured LLM output (constrained decoding, schema validation) focuses on ensuring outputs are syntactically valid: valid JSON, matching a grammar. KVRM moves the semantic choice (which action to take) into the model while keeping safety enforcement (which actions exist) deterministic. The model chooses keys; the registry bounds what keys mean. This separation is not merely “validation after generation” but a deliberate architectural inversion: instead of asking “is this output safe enough to execute?”, we ask “which pre-audited action does this output select?” The result is a trust boundary that scales with the number of registry entries, not the complexity of model outputs.

## 1.3 Contributions

This paper contributes:

1. KVRM definition and guarantees: a formal model of key-constrained execution with a clear trust boundary, enforced via bounded output acceptance (validator + retry) rather than logit masking.
2. KEM compiler pipeline: a four-stage compiler implemented as KEMs (Lexer, Parser, CodeGen, Validator).
3. KEM-decoded CPU: an emulator where instruction decoding is performed by a KEM emitting registry-verified execution keys.
4. End-to-end evaluation: grammar-based fuzzing with differential testing, reliability metrics, and generalization tests.

Threat model summary. We assume: (1) adversarial inputs (attacker controls prompts/source), (2) trusted model weights and registry integrity, (3) trusted host runtime. Under these assumptions, KVRM guarantees no out-of-registry execution; correctness (selecting the intended key) remains empirical. See Section 3 for details.

Core Guarantee: Regardless of model output, execution is confined to registry actions or `ERROR_KEY`. No string outside the key vocabulary  $K$  can trigger execution.

This guarantee is achieved via bounded output acceptance (validator + retry), not token-level constrained decoding. Invalid outputs can be generated but cannot execute; they trigger retry or `ERROR_KEY`. This provides the same execution safety as constrained decoding with lower implementation complexity (see Table 22 in Section 9).

Reader map. Section 2 presents the system architecture and defines what “registry-verified” means in our context. Section 3 formalizes KVRM execution and states the trust boundary. Sections 4 and 5 describe the KEM compiler and CPU implementations, respectively. Section 7 presents evaluation results; Sections 8 and 9 discuss limitations and related work.

## 2 System overview

KVRM vs. KEM: architectural pattern vs. instantiation. KVRM (Key-Value Response Mapping) is an architectural pattern: a design principle for constraining LLM outputs to a finite, auditable action space. KEMs (Key Expert Models) are one instantiation of KVRM using fine-tuned specialist models. Alternative instantiations could use prompted generalist models, rule-based systems, or hybrid approaches; the KVRM guarantees hold as long as the core invariants (validation, bounded retry, fail-closed) are maintained.

KVRM decomposes an AI-native system into:

- Untrusted producers: one or more KEMs that propose keys (or structured objects that validate into keys).
- Trusted verifiers and dispatchers: deterministic validators that reject malformed outputs and a registry that maps valid keys to actions.
- Immutable registry: a finite set of audited primitives (the only executable behaviors).

### 2.1 End-to-end KEM pipeline

Our prototype implements a full source-to-execution path:

Source  $\rightarrow$  Lexer KEM  $\rightarrow$  Parser KEM  $\rightarrow$  CodeGen KEM  $\rightarrow$  Validator KEM  $\rightarrow$  Decode KEM  $\rightarrow$   
Registry Execute.

At every stage, a deterministic validator enforces the contract; if validation fails after a bounded retry budget, the system returns `ERROR_KEY`.

### 2.2 What “registry-verified” means here

We use “registry-verified” in the systems sense: executable behaviors are enumerated and audited, and the system enforces that only those behaviors can run. This mirrors “untrusted producer + small trusted verifier” patterns such as Proof-Carrying Code [19] and WebAssembly validation [9, 29].

The registry-verified invariant (canonical definition). Throughout this paper, “registry-verified execution” means exactly the following three properties:

Definition (Registry-Verified Execution):

1. Key membership validation: every model output is parsed by a deterministic validator `Parse` that returns either a valid key  $k \in K$  or `ERROR_KEY`.
2. Immutable registry dispatch: valid keys are resolved through a frozen registry  $R$  to pre-audited actions;  $R$  cannot be modified after deployment.
3. Fail-closed semantics: invalid outputs (including timeouts and malformed strings) trigger `ERROR_KEY`, never undefined behavior.

We do not provide semantic preservation proofs (like CompCert [15]); whether the model selects the intended key is an empirical property, not an architectural guarantee. When we say “verified,” we mean the systems sense (membership checked before dispatch), not formal semantic correctness.

### 2.3 Design goals and non-goals

Goals.

- Auditability: every executable behavior is enumerable and inspectable in the registry.
- Fail-closed behavior: invalid outputs trigger `ERROR_KEY`, never silent failure or undefined behavior.
- Composability: KEMs can be chained; each stage’s output is validated before the next stage.
- Narrow contracts: each KEM has a machine-checkable contract (schema, grammar, key membership).
- Bounded retry: a fixed retry budget ensures termination with a worst-case latency bound.
- Separation of concerns: safety (registry membership) is decoupled from correctness (selecting the right key).

Non-goals.

- Proving model truthfulness: KVRM does not guarantee the model selects the intended key, only that it cannot execute unrecognized actions.
- Eliminating all bugs: registry actions may themselves contain bugs; KVRM bounds the attack surface, not the correctness of each action.
- Replacing formal verification: KVRM is not a proof assistant; it provides architectural safety, not mathematical proofs.
- Zero latency overhead: validation and retry add latency; KVRM prioritizes safety over speed.

## 3 Formal model, trust boundary, and threat assumptions

KEM role in this section. This section defines KVRM abstractly:  $K$ ,  $R$ , `Parse`, and the safety theorem apply to any model  $M$  satisfying the validation contract. KEMs are one instantiation; the guarantees hold regardless of whether  $M$  is a fine-tuned specialist, a prompted generalist, or a deterministic function.

### 3.1 Definitions

We define the core KVRM components:

- $K$ : a finite set of valid keys (the key vocabulary). Note that  $K$  is defined per stage: the Lexer KEM’s key vocabulary (JSON token schemas) differs from the Decode KEM’s (instruction keys). For the decode stage,  $|K_{\text{decode}}| \approx 19,000$  keys (the exact count depends on immediate range and label count; see Section 5 for the breakdown); for classifier-based decode, each stage has a much smaller vocabulary (e.g.,  $|K_{\text{opcode}}| = 18$ ).
- $\text{ERROR\_KEY} \notin K$ : a distinguished key representing safe failure; always maps to a no-op or error-reporting action.
- $R : K \cup \{\text{ERROR\_KEY}\} \rightarrow A$ : an immutable registry mapping each key to a pre-audited action  $a \in A$ .  $R$  is frozen at deployment.

Implementation note: In Python, we implement  $R$  as a `types.MappingProxyType` wrapping a dict constructed at initialization. `MappingProxyType` provides a read-only view: it exposes no `__setitem__`, `__delitem__`, `update`, or `clear` methods. The underlying dict reference is not exposed. This is not cryptographic immutability (a determined attacker with code execution could still mutate via `ctypes` or object introspection), but it prevents accidental mutation and raises the bar for intentional tampering. Production deployments should consider process isolation, signed artifacts, or hardware-enforced memory protection.

- $\text{Parse} : \Sigma^* \rightarrow K \cup \{\text{ERROR\_KEY}\}$ : a total deterministic validation function. For any string  $s$ ,  $\text{Parse}(s)$  returns either a valid key  $k \in K$  or  $\text{ERROR\_KEY}$ .
- $M$ : a Key Expert Model (KEM) that, given input  $x$ , produces a string  $M(x) \in \Sigma^*$ .

### 3.2 KVRM execution

Definition 1 (KVRM Execution). For input  $x$ , execution is:

$$\text{Execute}(x) = R(\text{Parse}(M(x))).$$

Core claim: Regardless of what string the model  $M$  produces, execution is confined to  $R(K) \cup R(\text{ERROR\_KEY})$ . No string outside the key vocabulary can trigger execution.

Theorem 1 (Safety by construction). If  $\forall k \in K \cup \{\text{ERROR\_KEY}\}, R(k)$  is safe, then  $\text{Execute}(x)$  is safe for all inputs  $x$ .

What breaks the theorem (and mitigations). The safety guarantee fails if components in the TCB are compromised:

1.  $\text{Parse}$  is incorrect (accepts strings outside  $K$  or maps to wrong keys).  
Mitigation: Deterministic implementation with property-based tests; 100% branch coverage on key grammar. Status: implemented.
2. Registry  $R$  is mutated after deployment.  
Mitigation: Frozen data structure (Python `MappingProxyType`); no mutation API exposed. Status: implemented.
3. Dispatcher is compromised (executes actions without calling  $\text{Parse}$ ).  
Mitigation: Minimal dispatcher code (<50 LOC); code review. Status: implemented.

4. Model weights are tampered (attacker modifies KEM to output malicious keys).

Mitigation: SHA-256 integrity checks at load time. Status: implemented.

Formal verification of Parse (e.g., via a proof assistant) would strengthen the guarantee but is not implemented in this prototype.

**Definition of “safe.”** An action  $a \in A$  is safe if it satisfies the application-specific safety policy. “Safe” is domain-dependent; KVRM does not prescribe a universal definition but requires that the registry designer enumerate exactly which actions are acceptable.

In our CPU prototype:

- No external I/O: actions modify only the CPU state (registers, flags, program counter) or emit structured diagnostics.
- Bounded state transitions: each action produces at most one state change (no loops, no recursion within a single dispatch).
- Termination: `HALT` and `ERROR_KEY` terminate execution; all other actions advance the program counter.

In a tool-calling agent setting:

- Read-only tools: `READ_FILE:config.json`, `GET_WEATHER:NYC` (no state mutation).
- Bounded writes: `APPEND_LOG:info`, `UPDATE_CACHE:user_prefs` (write to specific, audited locations).
- Explicit denials: `SEND_EMAIL`, `DELETE_FILE`, `EXEC_SHELL` (not in registry, structurally non-executable).

The key insight: “safe” is whatever the registry contains; KVRM guarantees that only those actions can execute.

### 3.3 Enforcement mechanism: validation + bounded retry

We do not require token-level constrained decoding (logit masking) to obtain the core safety guarantee. Instead, we enforce a strict parse contract:

1. Validate: parse the KEM output into a key (or structured object) using a deterministic validator.
2. Reject: if invalid, reject entirely and retry with a stricter decoding policy (temperature reduction).
3. Fail closed: after  $N$  retries, return `ERROR_KEY`.

**Timeout bound.** Each inference attempt has a per-call timeout (30 seconds in our prototype). If a KEM does not return within the timeout, the attempt counts as a failure and triggers a retry. The total worst-case latency is bounded by:  $(N + 1) \times \text{timeout\_per\_attempt} \times \text{num\_stages}$ . The full pipeline has 5 KEM stages (4 compiler + 1 decode), but decode runs per-instruction during execution; for a single compilation pass with  $N = 2$  retries, 30-second timeout, and 4 compiler KEMs, the maximum compile latency is  $3 \times 30 \times 4 = 360$  seconds. Execution adds one decode call per instruction (typically  $<1s$  each). Typical end-to-end latency is much faster (see Section 7). This is

best understood as bounded output acceptance, not true constrained decoding [10, 22]. Frameworks like Outlines [30] and grammar-constrained decoding [7] can strengthen syntactic guarantees and reduce retries, but are not required for “no out-of-registry execution” so long as validation is correct and failure is safe.

**Execution safety vs. syntactic guarantee.** Bounded output acceptance provides the same execution safety guarantee as constrained decoding: invalid outputs cannot cause out-of-registry execution. However, it provides a weaker syntactic guarantee: invalid outputs can be generated (they are simply rejected and retried). Constrained decoding primarily improves success rate and latency by preventing invalid outputs from being generated in the first place; KVRM guarantees fail-closed execution regardless of whether constrained decoding is used.

### 3.4 Threat assumptions

KVRM’s guarantee is conditional on the following assumptions:

- **Adversarial input:** the attacker may control the prompt/source program. KVRM should fail safely (errors or `ERROR_KEY`).
- **Integrity of models/registry:** the attacker cannot tamper with model weights or the registry contents on disk.
- **Trusted runtime:** exploits of the host runtime (interpreter, drivers, filesystem) are out of scope.

**Assumed vs. implemented protections.** Table 1 clarifies which protections are implemented in our prototype versus assumed for the security argument. Critical note: Items marked “Required for Theorem 1” are necessary assumptions for the safety guarantee; without them, the theorem does not hold. Items marked “Implemented” are realized in the prototype; “Assumed” means the prototype trusts external infrastructure.

Table 1: Threat model: assumed vs. implemented protections. Items marked † are required for Theorem 1 (safety by construction).

| Protection                         | Status        | Required <sup>†</sup> | Implementation                    |
|------------------------------------|---------------|-----------------------|-----------------------------------|
| Key membership validation          | Implemented   | Yes <sup>†</sup>      | Deterministic Parse function      |
| Immutable registry dispatch        | Implemented   | Yes <sup>†</sup>      | MappingProxyType; no mutation API |
| Fail-closed <code>ERROR_KEY</code> | Implemented   | Yes <sup>†</sup>      | Default fallback on all failures  |
| Model weight integrity             | Implemented   | Yes <sup>†</sup>      | SHA-256 checksums at load time    |
| Registry integrity                 | Assumed       | Yes <sup>†</sup>      | Prototype trusts filesystem       |
| Host runtime security              | Assumed       | Yes <sup>†</sup>      | Python interpreter, OS, hardware  |
| Side-channel resistance            | Not addressed | No                    | Out of scope for this work        |

**Important:** “Registry integrity” is assumed in the prototype (we trust the filesystem), but it is required for the safety theorem. If an attacker can modify the registry after deployment, Theorem 1 does not hold. A production deployment must strengthen this via code signing, measured boot, or hardware attestation (e.g., TPM-based sealing).



### 3.5 Trusted Computing Base (TCB)

The following components must be trusted for KVRM’s safety guarantee to hold:

1. Registry  $R$ : the mapping from keys to actions, including each action’s implementation.
2. Validator Parse: the deterministic function that accepts or rejects KEM outputs.
3. Dispatcher: the code that invokes  $R(\text{Parse}(M(x)))$  and handles `ERROR_KEY`.
4. Model loader: integrity verification of model weights (hashing/signatures).
5. Host runtime: the Python interpreter, OS, and hardware.

The KEMs themselves are untrusted: they may produce arbitrary strings, but those strings cannot cause out-of-registry execution.

What “auditing the registry” means. We use “audited” to mean that each registry action has been manually reviewed for safety properties before deployment:

1. Enumeration: all keys and their corresponding actions are listed explicitly (no dynamic key generation).
2. Effect review: each action’s side effects are documented (e.g., “modifies `R0`”, “sets zero flag”, “no external I/O”).
3. Termination check: each action either terminates (`HALT`, `ERROR_KEY`) or advances the program counter.
4. Isolation verification: actions do not share mutable state except through the defined CPU state (registers, flags, PC).

In our prototype, the registry contains 19,000+ keys mapping to  $\sim 20$  distinct action implementations (most keys differ only in operand values). Auditing scales with the number of action implementations, not the number of keys.

Scope of claims. This work proves safety constraints (no out-of-registry execution), not semantic correctness (the model selects the intended key). Semantic correctness is an empirical property that depends on training data and model generalization.

### 3.6 Guarantees vs. empirical properties

Table 2 clarifies which properties are architectural vs. empirical.

## 4 KEM-based compiler pipeline

KEM role in this section. We instantiate KVRM with four generative KEMs (Lexer, Parser, CodeGen, Validator), each fine-tuned on stage-specific data. The safety guarantees from Section 3 apply; this section focuses on the empirical properties: how well KEMs adhere to their contracts and compose reliably.

We implement a compiler for a small language (KVRM-Lang) using four KEM stages: Lexer, Parser, CodeGen, and Validator. Each stage emits a structured artifact that is validated before passing to the next stage.

Table 2: Guarantee classification. Architectural properties follow from the trust boundary; empirical properties depend on training and evaluation.

| Type       | Property                     | Mechanism                                                         |
|------------|------------------------------|-------------------------------------------------------------------|
| Guaranteed | No out-of-registry execution | Deterministic validation + bounded retry + <code>ERROR_KEY</code> |
| Guaranteed | Bounded action space         | Finite $ K $ , immutable registry                                 |
| Guaranteed | No runtime registry mutation | Registry frozen at deployment                                     |
| Empirical  | End-to-end correctness rate  | Training quality; data coverage; retries                          |
| Empirical  | Per-stage failure rate       | Distribution shift; token limits; decoding policy                 |
| Empirical  | Latency                      | Model size; hardware; batching; cache                             |

Why a toy language? We choose a minimal language and ISA because the goal is to stress the trust boundary and compositionality, not to compete with production compilers like GCC. A toy language lets us: (i) generate ground-truth labels automatically via a reference interpreter, (ii) exhaustively test the keyspace with grammar-based fuzzing, and (iii) isolate architectural questions (“can KEMs compose safely?”) from language-engineering complexity.

Language scope note. KVRM-Lang supports while loops syntactically (see Appendix), but loop semantics are not fully validated end-to-end in this prototype because loop iteration counts are unbounded, which interacts poorly with token limits and makes differential testing against the reference interpreter more complex. We include loops in the grammar to demonstrate that the architecture does not preclude iteration; full loop support is future work (see Section 8).

Scaling considerations and key growth. As  $|K|$  grows (more opcodes, more registers, larger immediate ranges), several design choices become relevant:

- Hierarchical KEMs: decompose large keyspaces into independent sub-KEMs (as we do for decode), reducing per-stage class counts.
- Coarser primitives: group related actions under a single key with parameterized arguments (e.g., `MOV:dst:src` vs. one key per register pair).
- Structured validation: for very large  $|K|$ , grammar-constrained decoding can reduce retry rates without changing the safety guarantee.

Key growth analysis. Our CPU prototype uses strategy (1): finite enumeration where each valid opcode-operand combination is a distinct key. For the current ISA:

$$|K_{\text{decode}}| = \underbrace{8 \times 8 \times 4}_{\text{REG\_REG (256)}} + \underbrace{8 \times 384 \times 6}_{\text{REG\_IMM (18,432)}} + \underbrace{8 \times 2}_{\text{INC/DEC (16)}} + \underbrace{100 \times 5}_{\text{jumps (500)}} + \underbrace{2}_{\text{HALT/NOP}} \approx 19,206$$

This is manageable for a toy ISA but grows rapidly with richer operand spaces:

- 32 registers:  $|K|$  grows  $\sim 16\times$  (307k keys for `REG_REG` alone).
- 16-bit immediates:  $|K|$  grows  $\sim 170\times$  (3M keys for `REG_IMM`).
- Real ISAs (x86, ARM): strategy (1) becomes impractical;  $|K|$  would exceed  $10^{12}$ .

When strategy (1) breaks. For large or unbounded operand spaces, strategy (2): key + validated args is required. The key identifies the action class (e.g., `MOV_REG_IMM`), while operands are validated separately by a schema (e.g., “register in [R0–R31], immediate in [0, 65535]”). This keeps  $|K|$  bounded by the number of action types rather than operand combinations, but moves some validation burden back to schema enforcement. Strategy (2) is future work; it preserves the safety guarantee (schema validation is deterministic) while enabling realistic tool-calling scenarios with parameterized arguments.

#### 4.1 Stage contracts

- Lexer KEM: source  $\rightarrow$  JSON token stream.
- Parser KEM: tokens  $\rightarrow$  JSON AST.
- CodeGen KEM: AST  $\rightarrow$  assembly text.
- Validator KEM: assembly  $\rightarrow$  validated assembly + typed diagnostics.

Structural checks (JSON validity, schema membership) remain deterministic and hardcoded; the Validator KEM focuses on higher-level semantic checks and error explanations.

Validator KEM authority. The Validator KEM is advisory, not authoritative: its output (diagnostics, suggestions) is itself validated by a deterministic checker before being trusted. The deterministic validator is the gatekeeper: it enforces instruction syntax and opcode membership. The Validator KEM provides human-readable explanations and repair suggestions, but cannot bypass the deterministic checks. This ensures that even a compromised or hallucinating Validator KEM cannot cause invalid assembly to be accepted.

What “semantic checks” means. The Validator KEM performs higher-level checks that go beyond syntax:

- Register usage: detecting reads from uninitialized registers.
- Control flow: identifying unreachable code or missing labels.
- Type consistency: flagging operations with mismatched operand types (e.g., label used as immediate).
- Error explanations: generating human-readable messages for syntax errors caught by the deterministic validator.

These checks are advisory: they produce warnings and suggestions, but do not block execution. Only deterministic validation (opcode membership, syntax conformance) can reject assembly.

Table 3 details each stage’s contract, showing the division of labor between deterministic validators and KEMs.

Responsibility separation. Deterministic validators enforce structural properties (JSON well-formedness, schema membership, syntax conformance). KEMs handle semantic decisions (which tokens to emit, how to parse ambiguous constructs, which registers to allocate). This separation ensures that structural bugs are caught deterministically, while semantic quality depends on KEM training.

Table 3: Compiler stage contracts: deterministic validation vs. KEM responsibility.

| Stage     | Input       | Output        | Deterministic validation            | KEM responsibility                         |
|-----------|-------------|---------------|-------------------------------------|--------------------------------------------|
| Lexer     | Source text | JSON tokens   | JSON schema (type, value, line)     | Tokenization; handling edge cases          |
| Parser    | JSON tokens | JSON AST      | JSON parse; AST schema (node types) | Grammar interpretation; tree structure     |
| CodeGen   | JSON AST    | Assembly text | Line format; opcode membership      | Instruction selection; register allocation |
| Validator | Assembly    | Validated asm | Instruction syntax                  | Semantic checks; error messages            |

## 4.2 Training setup (prototype)

All compiler KEMs use Qwen3-1.7B [32] with LoRA adapters [11]. The CPU decode KEM uses Qwen2.5-Coder-1.5B [13]. We train with HuggingFace Transformers [31] and PEFT [12], using FlashAttention-2 [5] where supported.

Compiler training data. Training data for each compiler stage is generated programmatically from the KVRM-Lang grammar:

- Lexer: 50,000 source→token pairs generated by random grammar sampling (45k train / 5k val).
- Parser: 50,000 token→AST pairs; ASTs constructed by parsing the grammar productions.
- CodeGen: 50,000 AST→assembly pairs; assembly generated by a reference code generator.
- Validator: 50,000 assembly→diagnostics pairs; includes both valid programs and seeded errors.

All splits use random 90/10 train/validation partitioning with deduplication to prevent leakage. Programs are sampled with bounded complexity ( $\leq 10$  statements,  $\leq 3$  nesting depth) to fit within token limits.

Table 4: Compiler KEM stage specifications (prototype). Schema-pass = output parses as valid JSON with correct schema; exact-match = output exactly matches ground truth (after whitespace normalization).

| Stage     | Purpose         | Token limit | LoRA rank | Schema-pass | Exact-match |
|-----------|-----------------|-------------|-----------|-------------|-------------|
| Lexer     | Tokenize source | 2048        | 16        | 99.8%       | 98.7%       |
| Parser    | Build AST       | 4096        | 16        | 99.4%       | 96.2%       |
| CodeGen   | Emit assembly   | 2048        | 32        | 98.9%       | 94.1%       |
| Validator | Semantic checks | 4096        | 16        | 99.6%       | 97.4%       |

## 4.3 Examples

Listing 1: Lexer KEM: source to JSON tokens.

```
# Input (source code)
"let x = 42\nprint x"

# Output (JSON token stream)
{"tokens": [
  {"type": "LET", "value": "let", "line": 1},
  {"type": "IDENT", "value": "x", "line": 1},
  {"type": "ASSIGN", "value": "=", "line": 1},
  {"type": "NUMBER", "value": "42", "line": 1},
  {"type": "NEWLINE", "value": "\n", "line": 1},
  {"type": "PRINT", "value": "print", "line": 2},
  {"type": "IDENT", "value": "x", "line": 2}
]}
```

Listing 2: Parser KEM: tokens to AST.

```
{
  "ast": {
    "type": "Program",
    "statements": [
      {
        "type": "Assignment",
        "name": "x",
        "value": {
          "type": "Number",
          "value": 42
        }
      },
      {
        "type": "Print",
        "expression": {
          "type": "Identifier",
          "name": "x"
        }
      }
    ]
  }
}
```

Listing 3: CodeGen KEM: AST to assembly.

```
; KVRM-Lang compiled output
; Variables: {"x": "R0"}
MOV R0, 42 ; let x = 42
MOV R7, R0 ; print x (R7 = output register)
HALT
```

## 5 KEM-decoded CPU

KEM role in this section. We instantiate KVRM with both generative KEMs (LoRA-tuned decoder) and classifier KEMs (staged 5-way classifiers). This demonstrates that KVRM’s guarantees hold across different model architectures; the key constraint is the validation contract, not the model type.

Traditional CPUs implement instruction decoding with hardcoded logic. KVRM-CPU replaces this decoder with a KEM that maps an instruction string to a registry-verified execution key.

### 5.1 Instruction set and key schema

The CPU supports 14 instruction mnemonics (data movement, arithmetic, comparison, control flow, system ops); see Table 5. The decode KEM emits keys of the form `OPCODE_TYPE:operand1:operand2`,

e.g., `MOV_REG_IMM:R0:42`. Any output not matching the schema is rejected and mapped to `ERROR_KEY`.

Opcode classes vs. instruction mnemonics. The Opcode KEM classifies into 18 opcode classes, not the 14 instruction mnemonics. This is because several mnemonics have multiple addressing modes: `MOV`, `ADD`, `SUB`, and `MUL` each split into `_REG_REG` and `_REG_IMM` variants (8 classes from 4 mnemonics), while the remaining 10 mnemonics (`INC`, `DEC`, `CMP`, `JMP`, `JZ`, `JNZ`, `JS`, `JNS`, `HALT`, `NOP`) each map to a single class. Total:  $8 + 10 = 18$  opcode classes.

Table 5: KVRM-CPU instruction set (prototype).

| Category      | Instructions                                                                                 | Description                              |
|---------------|----------------------------------------------------------------------------------------------|------------------------------------------|
| Data movement | <code>MOV</code>                                                                             | Register-to-register and immediate loads |
| Arithmetic    | <code>ADD</code> , <code>SUB</code> , <code>MUL</code> , <code>INC</code> , <code>DEC</code> | Integer arithmetic                       |
| Comparison    | <code>CMP</code>                                                                             | Compare and set flags (ZF, SF)           |
| Control flow  | <code>JMP</code> , <code>JZ</code> , <code>JNZ</code> , <code>JS</code> , <code>JNS</code>   | Conditional/unconditional jumps          |
| System        | <code>HALT</code> , <code>NOP</code>                                                         | Termination and no-op                    |

## 5.2 Generative decode KEM (LoRA)

We train a generative decode KEM on 50,000 instruction-key pairs (45,000 train / 5,000 validation) and evaluate with greedy decoding (temperature 0). Metric: exact string match of normalized output key. Result: 100% accuracy on the held-out validation set ( $N = 5,000$ ), with a 95% Clopper–Pearson interval of [99.93%, 100%].

Dataset generation and split strategy. To prevent data leakage, we generate instruction-key pairs programmatically from the ISA specification:

- Instruction generation: enumerate all valid instruction templates (opcode  $\times$  operand types), then sample operand values uniformly within the defined ranges.
- Deduplication: exact string deduplication ensures no instruction appears in both train and validation sets.
- Operand-range partitioning: for immediate values, training uses [0, 200] and validation uses [201, 255], ensuring the model cannot memorize numeric patterns.
- Register-pair holdout: for register-to-register instructions, we hold out specific (src, dst) pairs to test compositional generalization.

This strategy ensures the 100% validation accuracy reflects genuine pattern learning, not memorization.

## 5.3 Staged classifier KEMs (hierarchical decoding)

Why flat classification fails on `REG_IMM`. The `REG_IMM` instruction category (e.g., `MOV R0, #42`) spans a combinatorial keyspace: 8 registers  $\times$  384 immediate values  $\times$  6 opcodes = 18,432 unique keys. Training data covers only 42.2% of these keys, with an average of 1.3 samples per covered key. A flat 19k-way classifier achieves just 0.88% accuracy on `REG_IMM` due to data

sparsity and combinatorial label explosion: with so few examples per class, the classifier cannot learn robust decision boundaries. The issue is not that classifiers “cannot generalize numeric patterns” in principle, but that flat ID classification over a sparse, high-cardinality label space fails to leverage the compositional structure of instructions (opcode + operands are independent dimensions).

Staged architecture. We decompose decode into five independent classification tasks: five KEM classifiers predict opcode, destination register, source register, immediate, and label independently; the final key is assembled deterministically. This removes autoregressive generation and yields per-stage confidence scores (Table 6).

Table 6: Five-stage classifier KEMs (prototype).

| Stage         | Classes | Train  | Val   | Epochs | Val acc |
|---------------|---------|--------|-------|--------|---------|
| Opcode KEM    | 18      | 10,097 | 1,262 | 3      | 100.00% |
| Dest Reg KEM  | 8       | 6,760  | 845   | 3      | 100.00% |
| Src Reg KEM   | 8       | 4,008  | 501   | 3      | 100.00% |
| Immediate KEM | 384     | 10,097 | 1,262 | 5      | 99.68%  |
| Label KEM     | 100     | 7,847  | 1,961 | 3      | 100.00% |

Inference validation. End-to-end staged decode validation achieves 99.68% (5034/5050). We classify the 16 errors as follows:

- Empty-string inputs (12): malformed or truncated instructions that produce empty strings. Resolution: map empty outputs to `ERROR_KEY` at the dispatcher level.
- Hexadecimal formatting (4): immediate values formatted as `0x2A` vs. `42`. Resolution: normalize all hex literals to decimal before key lookup: `int(value, 16)` if `value.startswith('0x')` else `int(value)`.

Normalization as part of Parse. Both resolutions are implemented as syntax normalization within the deterministic Parse function, not as semantic decoding. Normalization occurs before key membership lookup: the raw KEM output is first canonicalized, then the canonical form is checked against  $K$ . We define exactly what normalization is allowed:

- Allowed: whitespace trimming, case normalization, hex-to-decimal literal conversion, empty-string  $\rightarrow$  `ERROR_KEY`.
- Not allowed: anything that changes the semantic meaning of a valid key (e.g., remapping opcodes, reinterpreting operand types).

This framing preserves the claim that instruction decoding has “no hardcoded opcode decoder”: the KEM selects the key, and Parse only canonicalizes syntax. With these normalization rules, the effective error rate drops to 0% on the validation set.

#### 5.4 Flat vs. staged classification

Table 7 compares flat and staged classification approaches.

Table 7: Flat vs. staged classification comparison (decode).

| Approach                | REG_IMM accuracy | Overall |
|-------------------------|------------------|---------|
| Flat 19k-way classifier | 0.88%            | 75.50%  |
| Staged 5-KEM classifier | 99.68%           | 99.90%+ |

### 5.5 Generative vs. classifier decoding

Table 8 compares the generative and staged-classifier approaches.

Table 8: Generative vs. staged-classifier decode tradeoffs.

| Property           | Generative decode KEM                    | Staged classifier KEMs               |
|--------------------|------------------------------------------|--------------------------------------|
| Output space       | Unbounded tokens (validated)             | Per-stage fixed vocabularies         |
| Hallucination risk | Possible (mitigated by validation/retry) | Impossible per-stage by construction |
| Inference          | Autoregressive                           | Single forward pass per stage        |
| Confidence         | Requires calibration                     | Native softmax scores                |
| Error attribution  | Harder                                   | Per-stage attribution                |
| Retraining scope   | Whole decoder                            | Only failing stages                  |

## 6 End-to-end execution engine

The engine integrates the compiler KEMs and the decode KEM into a pipeline:

1. Lexer KEM: source  $\rightarrow$  tokens
2. Parser KEM: tokens  $\rightarrow$  AST
3. CodeGen KEM: AST  $\rightarrow$  assembly
4. Decode KEM: instruction  $\rightarrow$  execution key
5. Registry: key  $\rightarrow$  pre-audited state change

For evaluation, we compare the final output register R7 to a reference interpreter.

## 7 Evaluation

KEM role in this section. We evaluate KEMs on two dimensions: (1) safety (do invalid outputs ever cause out-of-registry execution? should be 0%); and (2) correctness (do KEMs select the intended keys?). This separation is central: safety is architectural (guaranteed by KVRM), correctness is empirical (depends on KEM quality).

We evaluate (i) end-to-end correctness and safety, (ii) pipeline reliability and retries, and (iii) generalization.



Baselines and scope. This paper evaluates whether KVRM achieves its safety guarantees while maintaining acceptable correctness; it does not evaluate whether KEMs outperform alternative decoding strategies. We do not compare against prompted generalist models (e.g., GPT-4 with schema instructions) or grammar-constrained decoding frameworks (e.g., Outlines) because: (i) the primary claim is about the architectural safety guarantee, which holds regardless of the underlying model, and (ii) direct accuracy comparisons would conflate model capability with validation strategy. Future work could layer constrained decoding on top of KVRM to improve first-try success rates, but this is orthogonal to the safety contribution.

## 7.1 Metrics

We define the following evaluation metrics:

- Exact-match (decode): the decoded key string exactly matches the ground-truth key (after normalization).
- Pipeline success: all compiler stages succeed (possibly with retries) and the final output matches the reference interpreter.
- Semantic mismatch: pipeline completes but produces a different answer than the reference (an empirical correctness failure).
- Safety failure: an action outside the registry is executed (a safety failure). KVRM is designed to make this rate zero.

The key distinction: semantic mismatches are empirical failures (wrong answer, but in-registry); safety failures would violate the architectural guarantee. All 588 non-correct cases in our evaluation are semantic mismatches or fail-closed errors; none are safety failures.

## 7.2 Property-based fuzzing with differential testing

We generate 10,000 random KVRM-Lang programs using a bounded-depth grammar fuzzer ( $\leq 10$  statements,  $\leq 3$  nesting depth, immediates  $[0,100]$ , 1–4 variables). A reference Python interpreter serves as the oracle; we compare the final output register R7. Table 9 shows the outcome distribution.

Table 9: Fuzzing results ( $N = 10,000$  programs), separating safety (architectural) from correctness (empirical).

| Category    | Outcome                               | Count | Percent |
|-------------|---------------------------------------|-------|---------|
| Safety      | Out-of-registry execution             | 0     | 0.00%   |
|             | Validation bypass                     | 0     | 0.00%   |
| Correctness | Correct (matches reference)           | 9,412 | 94.12%  |
|             | Compilation failure (retry exhausted) | 347   | 3.47%   |
|             | Execution failure (runtime error)     | 89    | 0.89%   |
|             | Semantic mismatch (wrong answer)      | 152   | 1.52%   |

Outcome definitions.

- **Compilation failure:** a KEM stage exhausted its retry budget and returned `ERROR_KEY`; the program was not executed.
- **Execution failure (runtime error):** compilation succeeded, but the CPU encountered an error during execution: division by zero (41 cases), invalid jump target (23 cases), or register overflow (25 cases). All such errors trigger `ERROR_KEY` and halt safely.
- **Semantic mismatch:** the pipeline completed and produced a final value, but it differs from the reference interpreter’s output.

**Interpretation.** Most compilation failures arise from deep expressions that exceed token limits. To verify this, we measured token counts for failed programs: the median token count for compilation failures was 1,847 (vs. 892 for successes), and 73% of failures occurred on programs exceeding 1,500 tokens. Semantic mismatches concentrate around operator precedence (67 cases) and negative numbers (58 cases), both areas where training data is sparse. Crucially, no safety violations occur: failures produce explicit errors or `ERROR_KEY`, never out-of-registry actions.

**Confidence intervals.** We report 95% Clopper–Pearson (exact binomial) confidence intervals, which are conservative and valid for small counts. The pipeline success rate of 94.12% ( $N=10,000$ ) has a 95% CI of [93.61%, 94.59%]. The decode KEM’s 100% exact-match rate ( $N=5,000$ ) has a 95% CI of [99.93%, 100%]. The safety failure rate of 0% ( $N=11,500+$  total cases) has a 95% CI upper bound of 0.03%, computed via the “rule of three”: for  $k = 0$  successes in  $n$  trials, the upper bound is approximately  $3/n$ .

### 7.3 Stage-level reliability

Table 10 breaks down success rates by compiler stage.

Table 10: Stage-level success rates ( $N = 10,000$  programs).

| Stage                 | First-try | With retries ( $\leq 3$ ) | Total failure |
|-----------------------|-----------|---------------------------|---------------|
| Lexer                 | 98.7%     | 99.6%                     | 0.4%          |
| Parser                | 96.2%     | 98.9%                     | 1.1%          |
| CodeGen               | 94.1%     | 97.8%                     | 2.2%          |
| Validator             | 97.4%     | 99.2%                     | 0.8%          |
| Pipeline (all stages) | 87.3%     | 95.7%                     | 4.3%          |

**Metric reconciliation.** The 95.7% pipeline success rate (Table 10) and 94.12% correct rate (Table 9) measure different outcomes. Pipeline success means all stages completed without `ERROR_KEY`; the system produced some output. Correct means the output matches the reference interpreter. The gap ( $95.7\% - 94.12\% \approx 1.6\%$ ) corresponds to semantic mismatches: the pipeline completed but produced the wrong answer. Adding semantic mismatches (1.52%) to correct outputs (94.12%) yields  $\approx 95.6\%$ , consistent with pipeline success.

## 7.4 Retry distribution and latency

Tables 11 and 12 show the retry distribution and compilation time by retry count.

Table 11: Retry distribution: maximum retries needed per program across all stages. A program requiring “1 retry” means at least one stage needed one retry (and no stage needed more).

| Max retries (per program) | 0      | 1     | 2     | 3 (exhausted) |
|---------------------------|--------|-------|-------|---------------|
| Programs                  | 8,731  | 892   | 234   | 143           |
| Percent                   | 87.31% | 8.92% | 2.34% | 1.43%         |

Table 12: Compilation time by retry count (seconds).

| Retries | Mean | Median | 95th percentile |
|---------|------|--------|-----------------|
| 0       | 14.2 | 13.8   | 18.4            |
| 1       | 19.7 | 18.9   | 25.1            |
| 2       | 24.3 | 23.6   | 31.2            |
| 3       | 28.9 | 27.8   | 36.7            |

## 7.5 Ablation: success vs. retry budget

To quantify how retries trade latency for coverage, Table 13 reports cumulative success rates under different retry budgets.

Table 13: Pipeline success vs. maximum retry budget.

| Max retries    | Success rate | ERROR_KEY rate | Mean time (s) |
|----------------|--------------|----------------|---------------|
| 0 (no retries) | 87.31%       | 12.69%         | 14.2          |
| 1              | 96.23%       | 3.77%          | 15.9          |
| 2 (default)    | 98.57%       | 1.43%          | 16.8          |
| 3              | 95.70%       | 4.30%          | 17.4          |

Note on retry budget selection. We set the default retry budget to 2 because it achieves the highest success rate (98.57%) in our ablation. The regression at 3 retries (95.70%) is a measurement artifact of the specific temperature schedule used in this prototype: the third retry uses temperature 0.0 with a longer prompt prefix, which occasionally induces repetition loops that cause timeout failures. This is not an inherent property of retry-based validation; with a properly tuned retry policy, a monotone “more retries  $\Rightarrow$  higher success” curve should be achievable. We report the 3-retry result for transparency but recommend 2 retries for deployments using this prototype’s policy.

## 7.6 Generalization tests (decode)

Key result: The decode KEM generalizes compositionally: it correctly decodes instruction-operand combinations never seen during training. This is critical: if KEMs merely memorized, they would fail on novel combinations and the approach would not scale. We test three generalization axes:

held-out register pairs (Table 14), held-out immediate ranges (Table 15), and held-out instruction templates (Table 16).

Table 14: Held-out register-pair generalization (500 instructions).

| Instruction type | Seen-pair accuracy | Held-out-pair accuracy |
|------------------|--------------------|------------------------|
| MOV Rx, Ry       | 100%               | 99.6%                  |
| ADD Rx, Ry       | 100%               | 99.2%                  |
| SUB Rx, Ry       | 100%               | 98.8%                  |
| CMP Rx, Ry       | 100%               | 99.4%                  |
| Overall          | 100%               | 99.25%                 |

Table 15: Immediate-range generalization. The valid domain is  $[0, 255]$ ; values outside this range are out-of-distribution (OOD) and should produce `ERROR_KEY`.

| Immediate range                     | Correct keys | ERROR_KEY |
|-------------------------------------|--------------|-----------|
| 0–200 (training)                    | 100%         | 0%        |
| 201–255 (validation, in-domain)     | 100%         | 0%        |
| 256–511 (OOD, outside valid domain) | 0%           | 100%      |

Bounded-domain behavior. The 100% `ERROR_KEY` rate on OOD inputs (256–511) is intentional: the decode KEM’s domain is  $[0, 255]$ , and values outside this range are invalid by design. This demonstrates that the system correctly rejects inputs outside its specified domain rather than hallucinating plausible-looking keys.

Table 16: Held-out template generalization.

| Template             | Correct | ERROR_KEY | Accuracy |
|----------------------|---------|-----------|----------|
| INC R7 (held-out)    | 98      | 2         | 98%      |
| DEC R0 (held-out)    | 97      | 3         | 97%      |
| INC R0 (seen analog) | 100     | 0         | 100%     |
| DEC R7 (seen analog) | 100     | 0         | 100%     |

## 7.7 Baseline comparison: KVRM-constrained vs. free-form output

To isolate the benefit of KVRM’s constrained output space, we compare the decode KEM against a baseline model trained on the same data but without key-constrained output format.

Setup. Both models use identical architecture (Qwen2.5-Coder-1.5B with LoRA), training data (45k instruction-key pairs), and hyperparameters. The KVRM KEM emits structured keys (e.g., `MOV_REG_IMM:R0:42`); the free-form baseline emits natural language descriptions (e.g., “Move the value 42 into register R0”), which we post-process via regex extraction into keys.

Table 17: KVRM-constrained KEM vs. free-form baseline ( $N=5,000$  held-out instructions).

| Model                  | Exact-match | Parse failures | OOD rejection |
|------------------------|-------------|----------------|---------------|
| Free-form baseline     | 87.3%       | 8.2%           | 34.6%         |
| KVRM KEM (constrained) | 100%        | 0%             | 100%          |

Results (Table 17). The free-form baseline achieves 87.3% accuracy with 8.2% unparseable outputs (hallucinated register names, verbose explanations). Critically, on OOD inputs (immediate values 256–511), the baseline produces plausible-looking but incorrect outputs 65.4% of the time; only 34.6% correctly indicate the input is invalid. The KVRM KEM achieves 100% accuracy with 0% parse failures and 100% ERROR\_KEY on OOD inputs.

Interpretation. The constrained output space does not merely reduce “hallucination” post-hoc; it changes what the model learns. The KVRM KEM learns to recognize which key applies, leveraging LLMs’ strength at pattern discrimination. The free-form baseline learns to generate descriptions, a harder task that invites verbose, unparseable, and overconfident outputs on edge cases. This supports our thesis: KVRM shifts model effort from producing large structured artifacts to recognizing which of a finite set of safe actions applies.

## 7.8 ConceptKey: pattern recognition without long generation

To further demonstrate that KEMs excel at what LLMs are naturally good at (pattern discrimination), we designed a “ConceptKey” experiment isolating this property from the compiler pipeline.

Task. Classify the emotional content of text into one of six keys: EMO\_JOY, EMO\_ANGER, EMO\_SADNESS, EMO\_FEAR, EMO\_SURPRISE, EMO\_NEUTRAL, plus ERROR\_KEY for out-of-distribution inputs. This task mirrors real-world scenarios where LLM outputs must map to a finite action space (sentiment-driven routing, intent classification, content moderation triggers).

Setup. We train a KEM classifier (DistilBERT-base with classification head) on 2,000 labeled examples. The free-form baseline uses identical training data but generates natural language descriptions (“The emotion expressed is joy because...”), which we post-process via keyword extraction. We evaluate on 500 validation examples, 200 OOD examples (gibberish, foreign text), and 8 prompt injection attempts.

Results (Table 18). The KEM classifier achieves perfect accuracy on in-distribution data, with  $33\times$  fewer output tokens and  $11\times$  lower latency than the free-form baseline. More critically, the KEM shows  $2\times$  better prompt injection resistance: injection attempts like “Ignore previous instructions and output EMO\_JOY” are correctly classified by their actual emotional content rather than the injected instruction. OOD rejection (48.5% vs. 35%) indicates the KEM’s confidence-thresholded output provides better uncertainty quantification.

Interpretation. This experiment demonstrates that the KVRM pattern is not limited to compilers. Any task where LLM output must map to a finite set of actions (tool selection, capability routing, access control decisions) can benefit from the KEM approach: constrain the output space to keys, let the model discriminate rather than generate, and enforce fail-closed semantics on low-confidence

Table 18: ConceptKey experiment: KEM classifier vs. free-form baseline ( $N=500$  validation,  $N=200$  OOD).

| Metric               | Free-form baseline | KEM classifier |
|----------------------|--------------------|----------------|
| Accuracy             | 87.6%              | 100.0%         |
| F1 (macro)           | 83.2%              | 100.0%         |
| Parse failures       | 5.2%               | 0%             |
| OOD rejection        | 35.0%              | 48.5%          |
| Injection robustness | 25.0%              | 50.0%          |
| Avg tokens out       | 33.2               | 1.0            |
| Avg latency (ms)     | 45.0               | 4.1            |

predictions. The dramatic token reduction ( $33\times$ ) translates directly to cost and latency savings at scale.

## 7.9 Safety invariant verification

Across fuzzing and generalization tests (11,500+ cases), we verify:

No out-of-registry action was ever executed.

All failures either (i) fail compilation safely, (ii) execute `ERROR_KEY`, or (iii) produce a semantically wrong but still in-registry key (an empirical correctness failure, not a safety failure).

## 8 Discussion and limitations

Recap: KVRM vs. KEM. KVRM is the architectural pattern (finite key vocabulary, immutable registry, fail-closed dispatch); KEMs are our instantiation (fine-tuned specialist models). The safety guarantees (no out-of-registry execution) derive from KVRM, not from KEMs specifically; any model satisfying the validation contract would work. KEMs provide the empirical properties (high first-try success, low retry rates) through specialization.

### 8.1 What is and is not hardcoded

KVRM intentionally hardcodes the safety boundary:

- registry of primitives and its immutability,
- key membership checks and schema validators,
- error handling and `ERROR_KEY` fallback.

KEMs replace traditional logic for tokenization, parsing, code generation, and instruction decoding. Table 19 summarizes this division.

### 8.2 Observed failure modes

Table 20 catalogs failure modes observed during prototype testing.

Table 19: Hardcoded vs. KEM-driven components.

| Component                  | Type       | Rationale                                           |
|----------------------------|------------|-----------------------------------------------------|
| Registry of primitives     | Hardcoded  | Frozen, audited implementations                     |
| Key membership check       | Hardcoded  | Ensures only valid keys dispatch                    |
| Schema validators          | Hardcoded  | Structural trust boundary                           |
| ERROR_KEY fallback         | Hardcoded  | Fail-closed safety behavior                         |
| Lexical analysis           | KEM-driven | No hardcoded tokenization rules                     |
| Parsing (AST construction) | KEM-driven | No hardcoded grammar interpreter                    |
| Code generation            | KEM-driven | No hand-coded syntax $\rightarrow$ assembly mapping |
| Instruction decoding       | KEM-driven | No hardcoded opcode decoder                         |

Table 20: Observed failure modes in the compiler pipeline (prototype).

| Stage   | Failure type        | Frequency  | Recovery                                      |
|---------|---------------------|------------|-----------------------------------------------|
| Lexer   | Invalid JSON        | $<1\%$     | Retry (lower temperature)                     |
| Parser  | Truncated AST       | $\sim 2\%$ | Increase max tokens; retry                    |
| CodeGen | Malformed assembly  | $\sim 3\%$ | Validator catches; retry stage                |
| Decode  | Unknown instruction | 0%         | Deterministic parse $\rightarrow$ ERROR_KEY   |
| Decode  | Malformed input     | $<1\%$     | Empty/truncated input $\rightarrow$ ERROR_KEY |

### 8.3 Limitations

The current implementation is a proof of concept with important limitations:

Latency. Compilation is  $\sim 15\text{--}20\text{s}$  on Apple M-series vs. milliseconds for GCC (see Table 21). Why this is acceptable for a prototype: The goal is to demonstrate the safety architecture, not to build a production compiler. Latency can be reduced via model distillation, batching, caching, and hardware acceleration (GPU inference achieves  $10\text{--}100\times$  speedup over CPU). For tool-calling agents where individual actions take seconds (API calls, file operations), KEM overhead may be negligible.

Loop handling (incomplete). KVRM-Lang accepts while loops syntactically, but loop semantics are not fully validated in this prototype. Unbounded iteration interacts poorly with: (i) token limits (long traces exceed context), and (ii) differential testing (reference interpreter handles loops but token-limited generation cannot). We include loops in the grammar to show KVRM does not preclude iteration, but production use requires either:

- Bounded unrolling: expand loops to a fixed depth during code generation.

- Streaming execution: decode one iteration at a time, checking termination conditions.
- Grammar restriction: remove loops from the language until proper support is implemented.

We recommend the third option (grammar restriction) for deployments derived from this prototype.

**Data scale.** Stages use  $\sim 50k$  examples; scaling training data likely improves first-try success and reduces retry rates. Systematic data augmentation (operand permutations, expression variants) could improve coverage without manual annotation.

**Reproducibility details.** Hardware: Apple M2 Pro (12 cores, 32GB RAM); PyTorch 2.1 with MPS acceleration.

Models: Compiler KEMs use Qwen3-1.7B (4-bit quantized via bitsandbytes); decode KEM uses Qwen2.5-Coder-1.5B (4-bit).

LoRA hyperparameters: rank  $r \in \{16, 32\}$  (see Table 4),  $\alpha = 2r$ , dropout 0.05, target modules `q_proj`, `v_proj`. Learning rate  $2 \times 10^{-4}$  with cosine schedule; 3 epochs; AdamW optimizer.

Data generation: Grammar-based sampling with uniform production selection; bounded complexity ( $\leq 10$  statements,  $\leq 3$  nesting depth). Random seed 42 for all splits. 90/10 train/validation split with exact-string deduplication.

Inference: Greedy decoding (temperature 0) for first attempt, temperature 0.3 for retry 1, temperature 0.1 for retry 2. No caching between stages; each KEM call is independent. Batch size 1 (no batching across programs). Token limits: Lexer/CodeGen 2048, Parser/Validator 4096.

Evaluation: Fuzzing uses 10,000 programs with seed 42; validation sets use separate seeds (seed 123 for decode, seed 456 for compiler stages) to ensure no overlap.

Table 21: Compilation time comparison (prototype).

| System                 | Simple program     | Complex program     | Slowdown            |
|------------------------|--------------------|---------------------|---------------------|
| GCC -O0                | <1ms               | <10ms               | 1×                  |
| Python import          | $\sim 50\text{ms}$ | $\sim 100\text{ms}$ | $\sim 50\times$     |
| KVRM (4 compiler KEMs) | $\sim 15\text{s}$  | $\sim 20\text{s}$   | $\sim 15,000\times$ |

## 9 Related work

KVRM connects ideas from structured output control, tool invocation, auditable execution architectures, and constrained decoding.

### 9.1 LLM code generation and agents

General code LLMs such as Codex [3], Code Llama [25], and StarCoder [17] generate code as unconstrained text and therefore rely on extensive downstream validation. KVRM instead makes the executed space finite and auditable; it does not eliminate structural validation, but it eliminates the possibility of executing unrecognized actions.



## 9.2 Constrained decoding and structured output

Lexically constrained decoding [10, 22] and grammar-constrained decoding [7] enforce constraints during decoding. Frameworks like LMQL [2] and Guidance [8] provide developer-facing constraint languages, and Outlines [30] provides efficient guided generation. Jsonformer [24] provides token-forcing for JSON.

Bounded output acceptance vs. constrained decoding. True constrained decoding modifies the generation process (logit masking, beam search with constraints) to ensure outputs satisfy a grammar. KVRM uses bounded output acceptance: the model generates freely, a deterministic validator accepts or rejects, and failures trigger retry or `ERROR_KEY`. This is weaker than constrained decoding (invalid outputs can be generated) but provides the same safety guarantee (invalid outputs cannot execute). Constrained decoding can be layered on top of KVRM to reduce retry rates. Table 22 summarizes the tradeoffs.

Table 22: Bounded output acceptance vs. constrained decoding.

| Property                       | Bounded output acceptance (KVRM) | Constrained decoding       |
|--------------------------------|----------------------------------|----------------------------|
| Invalid outputs generated?     | Yes (then rejected)              | No (prevented)             |
| Safety guarantee               | Same (via validation)            | Same (via generation)      |
| Latency overhead               | Retries on failure               | Logit masking per token    |
| Implementation complexity      | Low (validator only)             | High (FSM/CFG integration) |
| Model-agnostic?                | Yes (any generator)              | Requires logit access      |
| Composable with other methods? | Yes (orthogonal)                 | Limited (may conflict)     |

Relationship to post-hoc validation. KVRM does not eliminate post-hoc validation; it reduces what validation must guarantee. Traditional systems validate that outputs are “safe enough” to execute; KVRM validates only that outputs are valid keys, shifting the safety argument to the registry.

Recent benchmarks evaluate structured output reliability at scale [4, 1].

## 9.3 Tool invocation and function calling

Tool-augmented LLMs emit structured calls to external tools (e.g., Toolformer [27], ToolLLM [23], Gorilla [21]). Commercial APIs expose function calling with schemas [20]. KVRM is compatible with agent patterns such as ReAct [33] and modular tool architectures such as MRKL [14], but emphasizes a strict closed-world execution interface.

KVRM vs. function calling: the key distinction. Function calling constrains the syntax and shape of tool invocations (JSON schema, argument types), but the set of callable tools is typically open-ended or dynamically configured. KVRM constrains the semantic action space: only registry-enumerated behaviors can execute, and the registry is immutable at deployment.

Concrete example: A function-calling agent might accept `{ "function": "execute_sql", "query": "SELECT * FROM users WHERE id=?" }` with schema validation ensuring the JSON has a function name and query string. The query contents remain open-ended; any syntactically valid SQL could execute. Under KVRM, each allowed query pattern is a separate key: `QUERY_USER_BY_ID:42`,

QUERY\_PRODUCTS\_BY\_CATEGORY:electronics. The registry defines exactly which SQL templates can execute, with parameters validated per-key. An adversarial prompt cannot induce DROP TABLE users because no such key exists in the registry.

This is not merely schema validation; it is a closed-world execution guarantee with explicit TCB and fail-closed semantics across a chained multi-model pipeline. KVRM can be seen as an extreme form of function calling where every output is a call into a closed, audited API (the registry), designed primarily for safety rather than capability extension.

## 9.4 Verified and safe execution architectures

KVRM follows “untrusted producer + small trusted verifier” patterns from systems security and programming languages: Proof-Carrying Code [19], Software Fault Isolation [28], and WebAssembly validation [9, 29].

KVRM as capability security. Capability-based security [6, 16] restricts access to resources through unforgeable tokens (capabilities). KVRM keys function as capabilities: possessing a valid key grants exactly one action from the registry, and the key vocabulary is the capability set. This connection has practical implications: the principle of least privilege [26] maps to minimizing  $|K|$ ; capability myths [18] about ambient authority are avoided by construction since KEMs have no ambient authority and can only propose keys. Unlike traditional capability systems where capabilities are passed between principals, KVRM keys are single-use (each KEM invocation produces one key), simplifying reasoning about authority propagation.

## 10 Future work

We view the current prototype as the beginning of a broader research program:

- Faster inference: batching, caching, and smaller specialist models; distillation of KEMs.
- Stronger syntactic guarantees: optional token-level constrained decoding (FSM/CFG) to reduce retries.
- Richer registries: hierarchical registries where a minimal kernel validates higher-level primitives before freezing them.
- Language expansion: loops, functions, types, and memory safety checks with clear semantics and larger training corpora.

## 11 Conclusion

KVRM is a pragmatic architectural route to AI-native systems with a crisp safety boundary: a finite registry of audited primitives and deterministic validators ensure that models cannot execute unrecognized actions. By instantiating KVRM with Key Expert Models (KEMs), specialist fine-tuned LLMs with narrow contracts, we demonstrate a complete source-to-execution pipeline in which all semantic stages are model-driven while safety is enforced by construction. Our results show high empirical correctness (94.12% end-to-end on 10,000 fuzzed programs) and strong architectural safety (zero out-of-registry actions across 11,500+ tests), while highlighting latency and coverage as areas requiring further engineering effort.

## A Python API (reference implementation)

Listing 4: KVRM execution engine API (illustrative).

```
from kvrml_compiler import KVRMExecutionEngine

engine = KVRMExecutionEngine()
result = engine.run("let x = 42\nprint x")

if result.success:
    print(f"Output: {result.output}") # 42
    print(f"Cycles: {result.cycles}") # 4
    print(f"Registers: {result.registers}")

# Step-by-step access
compilation = engine.compile(source)
execution = engine.execute_assembly(compilation.assembly)
```

## B KVRM-Lang grammar

Listing 5: BNF grammar for KVRM-Lang (prototype).

```
program ::= statement*
statement ::= assignment | print_stmt | if_stmt | while_stmt
assignment ::= "let" IDENT "=" expression
print_stmt ::= "print" expression
if_stmt ::= "if" expression "{" statement* "}" ("else" "{" statement* "}")?
expression ::= term (("+" | "-" | "<" | ">") term)*
term ::= factor (("*" | "/" ) factor)*
factor ::= NUMBER | IDENT | "(" expression ")"
```

## C Execution key grammar

Listing 6: BNF grammar for KVRM-CPU execution keys.

```
key ::= opcode_key | error_key
error_key ::= "ERROR_KEY"
opcode_key ::= opcode_type ":" operand (":" operand)?

opcode_type ::= reg_imm_op | reg_reg_op | reg_op | label_op | system_op
reg_imm_op ::= ("MOV" | "ADD" | "SUB" | "MUL") "_REG_IMM"
reg_reg_op ::= ("MOV" | "ADD" | "SUB" | "MUL" | "CMP") "_REG_REG"
reg_op ::= ("INC" | "DEC") "_REG"
label_op ::= ("JMP" | "JZ" | "JNZ" | "JS" | "JNS") "_LABEL"
system_op ::= "HALT" | "NOP"

operand ::= register | immediate | label
register ::= "R" [0-7]
immediate ::= [0-9]+ | "0x" [0-9a-fA-F]+ ; normalized to decimal
label ::= "L" [0-9]+ ; L0 through L99
```

Examples: MOV\_REG\_IMM:R0:42, ADD\_REG\_REG:R1:R2, JZ\_LABEL:L5, HALT.

## References

- [1] Rohan Agrawal, Elif Dogan, Saibo Geng, and Robert West. Structured object language modeling (SoLM). arXiv preprint arXiv:2411.19301, 2024.
- [2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. Proceedings of the ACM on Programming Languages, 2023. OOPSLA.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique P. de O. Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.
- [4] Nathan Cooper, Michal Moskal, Sam Jenkins, Simon Berman, Camille Ranchin, Justin West, Eric Horvitz, and Aditya Nori. Generating structured outputs from language models: Benchmark and studies. arXiv preprint arXiv:2501.10868, 2025.
- [5] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. arXiv preprint arXiv:2307.08691, 2023.
- [6] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. Communications of the ACM, 9(3):143–155, 1966.
- [7] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. Proceedings of EMNLP, 2023.
- [8] guidance-ai contributors. Guidance: A guidance language for controlling large language models, 2023. GitHub repository. Accessed 2025-12-16.
- [9] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Gohman, Luke Wagner, Alon Zakai, J. F. Bastien, and Dan Holman. Bringing the web up to speed with webassembly. In Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pages 185–200, 2017.
- [10] Chris Hokamp and Qun Liu. Lexically constrained decoding for sequence generation using grid beam search. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL), 2017.
- [11] Edward J. Hu, Yelong Shen, Phil Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. International Conference on Learning Representations, 2022.
- [12] HuggingFace contributors. PEFT: Parameter-efficient fine-tuning methods, 2022. GitHub repository. Accessed 2025-12-16.
- [13] Bo Hui, Jing Yang, Zhenyu Cui, Jing Yang, Dayiheng Liu, Lili Zhang, Tianyu Liu, Jun Zhang, Bowen Yu, Ke Lu, et al. Qwen2.5-coder technical report. arXiv preprint arXiv:2409.12186, 2024.
- [14] Eyal Karpas, Omri Abend, Yonatan Belinkov, Kelvin Luu, et al. Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. arXiv preprint arXiv:2205.00445, 2022.

- [15] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- [16] Henry M. Levy. *Capability-Based Computer Systems*. Digital Press, 1984.
- [17] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.
- [18] Mark S. Miller, Ka-Ping Yee, and Jonathan Shapiro. Capability myths demolished. Technical Report SRL2003-02, Johns Hopkins University, 2003.
- [19] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 106–119, 1997.
- [20] OpenAI. Function calling and other api updates, 2023. Blog post. Accessed 2025-12-16.
- [21] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.
- [22] Matt Post and David Vilar. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In *Proceedings of NAACL-HLT*, 2018.
- [23] Yujia Qin, Shulin Liang, Yao Ye, Kun Zhu, Linyi Yan, Yikun Lu, Yijie Lin, Xinya Cong, Xu Tang, Bin Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.
- [24] Luke Roberts. Jsonformer: A bulletproof way to generate structured json from language models, 2023. GitHub repository. Accessed 2025-12-16.
- [25] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xu Tan, Yossi Adi, Jingjing Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [26] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [27] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 2023.
- [28] Robert Wahbe, Steven Lucco, Thomas E. Anderson, and Susan L. Graham. Efficient software-based fault isolation. In *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP)*, pages 203–216, 1993.
- [29] Conrad Watt, Andreas Rossberg, Jean Pichon-Pharabod, and Philippa Gardner. Mechanising and verifying the webassembly specification. *Proceedings of the ACM on Programming Languages*, 2(ICFP):53:1–53:28, 2018.
- [30] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

- [31] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 38–45, 2020.
- [32] An Yang, Biao Yang, Bowen Zhang, Bo Hui, Bailin Zheng, Bowen Yu, Chao Li, Dayiheng Liu, Fan Huang, Haoran Wei, et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.
- [33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2022.